VisionQ — Setup & User Guide

Installation, configuration, and basic usage

---

1. System Requirements · 2. Installation · 3. First Login · 4. Detection Services · 5. TCP Push · 6. API Tokens · 7. Recordings · 8. Updating · 9. Troubleshooting

1 — SYSTEM REQUIREMENTS

Component	Minimum	Recommended
OS	Ubuntu 20.04	Ubuntu 22.04 LTS
CPU	4 cores	8+ cores
RAM	4 GB	16 GB
Storage	20 GB	100 GB+ (recordings)
GPU	None — CPU mode	NVIDIA GPU, 4 GB+ VRAM

> A GPU is not required. Without one, VisionQ runs in CPU mode at roughly 2–8 FPS — sufficient for slow camera streams such as vehicle gates. The installer detects your hardware automatically.

2 — INSTALLATION

Copy the `visionq/` folder to your server and open a terminal inside it. Then run:

```
bash setup.sh
```

The script handles everything automatically:

1. Installs Docker (if not already present)
2. Detects your GPU and installs the NVIDIA Container Toolkit if needed
3. Generates a secure secret key and creates the `.env` config file

4 Builds and starts VisionQ

The first build takes **5–20 minutes** depending on internet speed. When finished, the script prints the server address. Open it in any browser:

```
http://<server-ip>:8000
```

> The installer is safe to run multiple times. Re-running it on an already-installed system will skip completed steps and just start the container.

3 — FIRST LOGIN & SECURITY SETUP

Two accounts are created automatically on first startup:

```
Username    Password       Role
admin       admin123       Full access
customer    customer123    View only
```

> **Important:** Change both passwords immediately after first login. Default passwords must not be left in place on a production system.

To change a password: log in → click the gear icon (⚙) in the top right → **Settings** → **Change Password**.

Account roles

Role	Permissions
admin	Full access — manage services, settings, users, API tokens, all data
customer	Read-only — view live results and recordings only

4 — DETECTION SERVICES

A detection service is one camera or video source. You can run multiple services simultaneously.

1 Go to **Detection** from the main menu

2. Click **Add Service** (+)
3. Fill in the service details (see table below) and click **Save**
4. Click **Start** (▶) on the service card to begin detection

Field	Description	Example
Name	Unique identifier for this camera	`gate-camera-1`
Model	Path to the YOLO model file	`/villapro-data/models/best.pt`
Input mode	How frames are received	RTSP / HTTP / Video file
RTSP URL	Camera stream address	`rtsp://192.168.1.100/stream`
Confidence	Minimum detection confidence (0–1)	`0.5`
Auto-start	Restart automatically when server reboots	Recommended: enabled

> Model files must be placed in `/villapro-data/models/` on the server. Upload them via the Files section in the UI or copy them with SCP/SFTP.

5 — TCP PUSH

VisionQ can push every detection result in real time to your backend system over TCP or SSL. This is the primary way to integrate VisionQ with external applications.

How it works

- Your system connects to VisionQ on a configured port
- VisionQ sends one JSON message per line on every detection
- A keepalive `{"type":"keepalive"}` is sent every 5 seconds when idle
- Multiple clients can connect at the same time

Enabling TCP push

1. Open a service card → **TCP Push tab**
2. Enter a port number (e.g. `9001`) and click **Start**
3. Add the port to `docker-compose.yml` under `ports` : `- "9001:9001"`

Settings are saved and TCP push restarts automatically on server reboot.

Example — receiving results in Python

```
import socket, json

sock = socket.create_connection(("visionq-server-ip", 9001))
buf = ""
for chunk in iter(lambda: sock.recv(4096).decode(), ""):
    buf += chunk
    while "\n" in buf:
        line, buf = buf.split("\n", 1)
        msg = json.loads(line)
        if msg.get("type") != "keepalive":
            print(msg)
```

6 — API TOKENS

API tokens allow external systems to access the HTTP API without a username/password login. Tokens never expire and are meant for machine-to-machine access.

1. Log in as admin → **Settings → API Tokens**
2. Enter a label (e.g. `backend-server`) and click **Create**
3. Copy the token — it is shown **once only** and cannot be retrieved later

Use the token in every request:

```
Authorization: Bearer <your-token>
```

For SSE streams, pass it as a query parameter instead:

```
GET /api/detection/services/camera1/stream?token=<your-token>
```

To revoke a token: Settings → API Tokens → **Revoke**.

7 — RECORDINGS & DEBUG LOGS

Recordings

Enable stream recording from the service card → **Recording tab** → **Start Recording**. Files are saved to `/villapro-data/recordings/` and can be browsed, downloaded, or deleted from **Recordings** in the main menu.

Debug logs

Enable CSV debug export from the service card → **Debug tab** → **Start Debug Log**. Files are saved to `/villapro-data/debug/` and can be managed from **Debug Logs** in the main menu.

8 — UPDATING

To update to a new version, copy the updated files to the server and run:

```
docker compose build && docker compose up -d
```

All data is stored in the `villapro-data/` volume and is never affected by updates.

Action	Command
Stop	`docker compose down`
Start	`docker compose up -d`
View logs	`docker compose logs -f`
Restart	`docker compose restart`
Update	`docker compose build && docker compose up -d`

9 — TROUBLESHOOTING

Problem	Solution
Cannot open the web interface	Run `docker compose logs -f`. Check that port 8000 is not blocked: `sudo ufw allow 8000`
GPU not used / detection is slow	Run `nvidia-smi` to verify drivers. Re-run `bash setup.sh` to reinstall the NVIDIA Container Toolkit.
Service fails to start	Check the service log in the UI. Common causes: wrong model path, camera unreachable, wrong RTSP credentials.
RTSP not connecting	Test from the server: `ffprobe rtsp://<url>`. Check camera credentials and firewall.
Forgot admin password	Stop the container, delete `villapro-data/visionq.db`, restart. Users reset to defaults — all other data is kept.
Port already in use	Change `VISIONQ_PORT` in `.env` and run `docker compose up -d`.